

Field and Service Robotics
Technical Report

DRIVING AND POINTING

Multi-Controller Approaches for Car-Like Robots with Articulated Arms

Prof. Fabio Ruggiero

Baiano Alessia P38000241
Mancinelli Bruno Andrea P38000298

GitHub repository: [Click here](#)

Simulation videos: [Click here](#)

Table of Contents

Introduction	1
1 Kinematic model	3
2 Trajectory planning	5
3 Controllers and Odometry	8
3.1 Control via approximate linearization	8
3.2 Input-output linearization via static feedback	10
3.3 Full-state linearization via dynamic feedback	11
3.4 Odometry and SLAM	13
4 Comparison and discussion of controllers' performance	14
4.1 Control via approximate linearization results	14
4.2 Input-Output linearization via static feedback results	16
4.3 Full-state linearization via dynamic feedback results	18
5 Look-At-Point behavior in manipulator control	21
5.1 Results	23

Introduction

Autonomous mobile robotics has become a cornerstone in many fields of engineering, ranging from industrial automation to service and field robotics. Among the various robotic platforms, car-like robots, also known as Ackermann-steered, are widely studied due to their direct analogy with real vehicles and their non-holonomic motion constraints, which make trajectory planning and control particularly challenging.

This technical report presents the implementation and evaluation of three different controllers for a car-like mobile manipulator. The experimental platform consists of the RB-VOGUI mobile base by Robotnik Automation (Figure 1), equipped with a five-degree-of-freedom (DoF) articulated robotic arm, specifically the UR5 by Universal Robots (Figure 2).

The decision to combine this particular mobile base with the UR5 manipulator was motivated by their widespread use together in real-world industrial and research applications. Such a combination allows for versatile mobile manipulation tasks, where the robot can navigate complex environments and perform precise manipulation actions. To accurately represent this integrated system in simulation, we connected the URDF models of the mobile base and the robotic arm, thereby ensuring a coherent and realistic kinematic description. This approach enables us to reproduce the interactions between the mobile platform and the manipulator, and to develop control strategies that reflect practical scenarios encountered in real deployments.



Figure 1: RB-VOGUI.

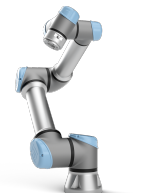


Figure 2: UR5.

The primary focus of this work is twofold: first, the analysis and comparison of multiple control strategies for accurate trajectory tracking of the mobile base; second, the development and implementation of a "look-at-point" behavior for the manipulator, enabling the robot to orient its end-effector toward the position of an ArUco marker while the base is in motion.

To address the trajectory tracking problem, we implemented and tested three different control strategies: a controller based on approximate linearization, a controller based on input-output linearization via static feedback, and a controller based on full-state linearization via dynamic feedback.

For the manipulation task, we first implemented a position controller to move the robotic arm into a desired configuration. Once the end-effector reaches this initial pose, the system switches to a velocity controller whenever an ArUco marker is detected. This enables the look-at-point behavior, allowing the manipulator to continuously adjust the orientation of the end-effector so that it remains directed toward the target marker, even as the mobile base follows its planned trajectory. Such capability is essential in applications such as visual servoing, inspection, and human-robot interaction, where it is crucial to maintain focus on a moving or changing target.

The structure of this report is organized as follows.

In the first section, we present the kinematic model of the mobile base, with a particular focus on the equivalent bicycle model for Ackermann-steered vehicles, which provides an intuitive yet accurate representation of the system's non-holonomic constraints.

The second section describes the trajectory planner, detailing the algorithms employed to generate feasible and smooth paths for the robot in complex environments.

The third section delves into the implementation of the control strategies for the mobile base. Here, we provide an in-depth discussion of each controller, outlining the theoretical background, the practical implementation, and the integration with the trajectory planner. Additionally, we introduce the use of SLAM (Simultaneous Localization and Mapping) techniques for odometry correction, highlighting their importance for maintaining accurate localization in real-world scenarios.

In the fourth section, we present and analyze the experimental results, offering a critical comparison of the performance of the different controllers across various test cases. This analysis includes both quantitative metrics and qualitative observations regarding stability, accuracy, and robustness.

The fifth section focuses on the implementation of the look-at-point behavior for the manipulator. We detail the development of the velocity controller for the arm, discuss its integration with the mobile base, and demonstrate its effectiveness in dynamically tracking visual targets during motion.

Finally, the concluding section summarizes the comparative evaluation of the control strategies, highlighting their respective strengths and limitations, and discusses the practical implications for trajectory tracking and visual servoing in dynamic environments.

All the components of this project have been developed and integrated using the Robot Operating System 2 (ROS 2) framework, which provides a flexible and modular environment for robotic software development. The simulation of the robotic platform and its environment was performed in Gazebo, allowing for comprehensive testing and validation of both navigation and manipulation tasks. The control algorithms and supporting software were implemented in both Python and C++, leveraging the strengths of each language for rapid prototyping and high-performance execution. This combination of modern robotic middleware, realistic simulation, and multi-language development ensured a robust and extensible workflow throughout the project.

The `ros2_kdl_package` was used for the implementation of the controllers, providing an interface with the Kinematics and Dynamics Library (KDL). KDL enables efficient handling of 3D frame transformations, vector and twist operations, and supports both kinematics and dynamics computations for kinematic chains and trees. The `ros_vision` package was integrated for the detection of ArUco markers as part of the vision system developed in this project. The design started from the official URDF of the RBVogui mobile base, which was then extended to include a UR5 manipulator. The UR5 was selected following a careful analysis of compatibility and performance requirements. The manipulator was integrated using the `Universal_Robots_ROS2_Description` package. Both URDF models used are official open-source versions, publicly available on the GitHub repositories of their respective manufacturers.

However, the entire process of integration, customization, and coordination between the two robotic systems (mobile base and manipulator) was fully developed by the authors. From a theoretical standpoint, the controller design was based on selected academic references (references [1] and [2]) with the goal of implementing realistic controllers suited to the car-like configuration of the robot. The chosen approach was structured to ensure consistency between the kinematic/dynamic model and the desired system behavior, with particular attention to theoretical coherence with the Ackermann model. In this context, a preliminary study of the main available libraries was also conducted to identify the most suitable solution for managing Ackermann control in the ROS 2 environment.

The implementation of the “look-at-point” behavior was inspired by a previous project developed by the same authors, but it was revised to fit a completely different type of task, characterized by specific constraints and integrated into a more complex functional context.

Ultimately, beyond the use of open-source robotic models, all the integration, control, planning, and coordination work — defined within the `rbvogui_description` package — was entirely designed, developed, and validated autonomously by the authors.

1 Kinematic model

The Ackermann steering geometry is a fundamental concept in the modeling of car-like vehicles. It captures the essential kinematic constraints imposed by the steering mechanism, which allows only the front wheels to steer, while the rear wheels simply follow the trajectory.

Typically, only one pair of wheels is driven, with the rear wheels most commonly serving as the drive wheels, providing the forward propulsion for the vehicle. This configuration (Figure 4) is widely used in both real-world vehicles and autonomous mobile robots due to its simplicity and effectiveness in various environments.

The vehicle’s motion is described in a local body-fixed reference frame, with its origin typically at the center of mass, the x-axis pointing forward, the y-axis pointing to the left, and the z-axis upward (according to the ISO convention).

In our implementation, we chose to locate the body-fixed reference frame at the midpoint between the rear wheels, rather than at the vehicle’s center of mass (Figure 3). This choice is motivated by practical considerations: placing the origin at the rear axle center simplifies the kinematic equations and ensures that the odometry provides the state variables (position and orientation) required for the control of the mobile base. This convention is widely adopted in mobile robotics, as it aligns the reference frame with the point used for trajectory tracking and controller design.

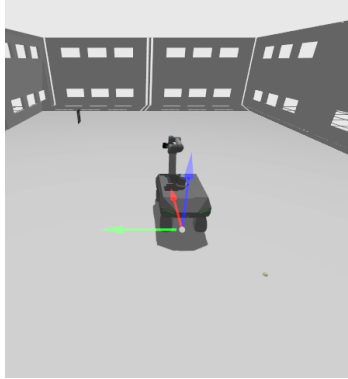


Figure 3: Simulation environment with the car-like mobile manipulator and its body-fixed reference frame (ISO convention).

A crucial kinematic constraint for Ackermann vehicles is the non-holonomic constraint: the vehicle cannot move sideways; it can only move in the direction it is facing, with its turning determined by the steering angles.

In classical Ackermann steering, the left and right front wheels are steered at different angles, ensuring that all wheels roll without lateral slip and their rolling axes intersect at the ICR.

However, as highlighted by Pesterev [2], the full four-wheel geometry can be further simplified for control purposes by introducing an equivalent single steering angle δ , as in the bicycle model (or single-track model, see Figure 5). In this abstraction, the two front wheels are replaced by a single virtual wheel located at the center of the front axle, and the same for the rear axle. This simplification is valid when the left and right wheels behave similarly.

In this model, the geometric relationship between the steering angle δ and the instantaneous curvature k of the path followed by the vehicle is given by:

$$\tan(\delta) = kL \tag{1}$$

where L is the wheelbase of the vehicle. This equation allows the complex four-wheel steering geometry to be replaced by a single steering angle that produces the same curvature, greatly simplifying both modeling and control design.

According to Ackermann steering theory, there is a specific geometric relationship that links the steering angle δ of the equivalent single front wheel (as in the bicycle model) to the steering angles of the individual left and right front wheels (δ_i and δ_e). Knowing that $\tan \delta_1 = \frac{\kappa L}{1 - \kappa H/2}$, $\tan \delta_2 = \frac{\kappa L}{1 + \kappa H/2}$, where H is the distance between the left and right front wheels, this relationship can be expressed as:

$$\tan(\delta) = \frac{2 \tan(\delta_e) \tan(\delta_i)}{\tan(\delta_e) + \tan(\delta_i)} \quad (2)$$

where:

- δ is the steering angle of the equivalent wheel in the single-track (bicycle) model,
- δ_e and δ_i are the steering angles of the exterior and interior front wheels, respectively.

However, this formula exhibits a singularity when the angles are close to zero. In such cases, when the angles are sufficiently small, a good approximation that can be used is:

$$\delta \approx \frac{\delta_i + \delta_e}{2}$$

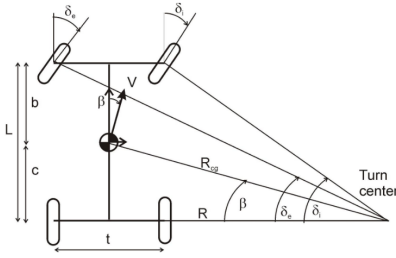


Figure 4: Ackermann model

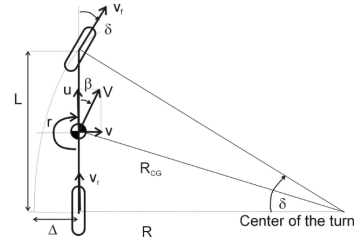


Figure 5: Bicycle model

In the case where the rear wheels are the drive wheels, the most widely adopted equivalent kinematic model for a car-like robot is the following bicycle model:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \\ \frac{\tan \phi}{L} \\ 0 \end{bmatrix} v + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \omega \quad (3)$$

where (x, y) is the position of the reference point (typically at the center of the rear axle), θ is the orientation of the vehicle (yaw angle), v is the linear velocity applied at the rear wheels, L is the wheelbase, and δ is the steering angle of the front wheel.

There is a singularity in the model when $\phi = \pm \frac{\pi}{2}$, at which point the first vector field exhibits a discontinuity. This situation corresponds to the case where the front wheel is perpendicular to the vehicle's longitudinal axis, causing the car to become immobilized or "jammed."

The system is subject to two nonholonomic constraints, one for each wheel:

$$\begin{cases} \dot{x} \sin(\theta + \phi) - \dot{y} \cos(\theta + \phi) - \dot{\theta} \ell \cos \phi = 0 \\ \dot{x} \sin \theta - \dot{y} \cos \theta = 0 \end{cases} \quad (4)$$

with x_f, y_f denoting the cartesian coordinates of the front wheel.

In the context of nonholonomic mobile robots, it is often beneficial to transform the original kinematic model into a canonical form, such as the chained form. This transformation provides a standardized structure that greatly facilitates both the analysis and the design of control strategies.

In our work, we leveraged the chained form representation in the development of both our first and third controllers, enabling more systematic and effective control law synthesis.

The state and input variables can be transformed as follows to obtain the chained form for the car-like robot:

$$\begin{cases} x_1 = x \\ x_2 = \frac{\tan \phi}{l \cos^3 \theta} \\ x_3 = \tan \theta \\ x_4 = y \end{cases} \quad (5)$$

with the input transformation:

$$\begin{cases} v_1 = \frac{u_1}{\cos \theta} \\ v_2 = -3 \frac{\sin \theta \sin^2 \phi}{l \cos^2 \theta} \cdot u_1 + l \cos^3 \theta \cos^2 \phi \cdot u_2 \end{cases} \quad (6)$$

In this transformed system, the car-like robot kinematics assume the following chained form:

$$\begin{cases} \dot{x}_1 = u_1 \\ \dot{x}_2 = u_2 \\ \dot{x}_3 = x_2 u_1 \\ \dot{x}_4 = x_3 u_1 \end{cases} \quad (7)$$

All these transformations are better explained in [1].

2 Trajectory planning

The trajectory planning methodology adopted in this work is based on the approach developed in Homework 2 of the Field and Service Robotics course.

The main objective is to generate a smooth and dynamically feasible reference trajectory for the car-like mobile base, while satisfying all the constraints of the system.

The path is constructed using cubic polynomials for $x(s)$ and $y(s)$, with $s \in [0, 1]$. The coefficients of these polynomials are determined so that the path starts and ends at prescribed positions and orientations, satisfying the necessary boundary conditions.

This cubic parameterization is chosen because it provides enough flexibility to satisfy the constraints on both position and heading at the endpoints, while reducing the risk of oscillations or numerical instability associated with higher-order polynomials or splines.

To obtain the necessary derivatives (such as velocity, acceleration, and higher-order terms), numerical gradients are computed instead of using lengthy analytic expressions. This approach improves the readability and reliability of the code.

It is important to note that numerical differentiation inevitably introduces approximation errors and tends to amplify noise or irregularities in the input data. This issue can be particularly critical in the case of the full state feedback linearization controller, which requires derivatives up to the third order.

To mitigate these effects and improve the smoothness of the reference signals, a low-pass filter is applied to the first, second, and third derivatives of position. Specifically, an exponential filter is used, which follows the recursive relationship:

$$y_k = \alpha x_k + (1 - \alpha)y_{k-1} \quad (8)$$

The filter coefficients α are chosen based on the order of the derivative:

- $\alpha = 0.7$ for the first derivative,
- $\alpha = 0.5$ for the second derivative,
- $\alpha = 0.1$ for the third derivative.

This decreasing choice of α reflects the fact that higher-order derivatives tend to be more sensitive to noise and thus require stronger filtering (i.e., a smaller α , resulting in a slower response). However, applying a low-pass filter inevitably introduces a delay in the system's response, so the filter parameters must be carefully tuned to balance noise suppression and responsiveness.

The orientation $\theta(s)$ along the path is computed as:

$$\theta(s) = \arctan 2 \left(\frac{dy}{ds}, \frac{dx}{ds} \right) \quad (9)$$

and its derivatives are obtained numerically in the same way.

A key element of the planning procedure is the time parameterization, which is performed using a cubic law:

$$s(t) = 3 \left(\frac{t}{T} \right)^2 - 2 \left(\frac{t}{T} \right)^3, \quad (10)$$

where T is the total trajectory duration. This law ensures smooth starting and stopping of the robot, as both $s(0)$ and $s(T)$, and their derivatives, are zero at the endpoints. The temporal derivatives such as $\dot{s}(t)$ are computed analytically, which allows mapping spatial derivatives into the time domain.

Using these relationships, the time-dependent reference signals are constructed as follows:

$$v(t) = \sqrt{\left(\frac{dx}{ds} \right)^2 + \left(\frac{dy}{ds} \right)^2} \cdot \dot{s}(t) \quad (11)$$

$$\omega(t) = \frac{d\phi}{ds} \cdot \dot{s}(t) \quad (12)$$

$$\phi(t) = \arctan \left(\frac{L \frac{d\theta}{ds}}{\sqrt{\left(\frac{dx}{ds} \right)^2 + \left(\frac{dy}{ds} \right)^2}} \right) \quad (13)$$

To guarantee feasibility, an adaptive time-scaling strategy is implemented. If any of the reference signals (linear velocity, angular velocity, or steering angle) exceeds the prescribed limits, the total duration T is increased, and the reference profiles are recalculated. This process is repeated until all constraints are satisfied.

Notice that, since the time and path discretizations are not generally aligned, all reference signals and their derivatives are interpolated to ensure they are available at the desired time instants. This interpolation step is crucial to maintain the smoothness and accuracy of the profiles, and to provide consistent data to the controllers.

All the reference signals are exported in tabular (CSV) format for easy integration with simulation and control environments.

To evaluate and compare the performance of the different controllers, a reference trajectory, depicted in Figure 6, was generated and used as a benchmark for all simulations.

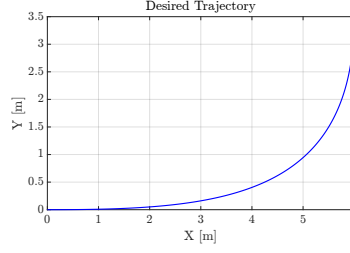


Figure 6: Desired trajectory used for the experimental validation of the controllers.

The shape and complexity of this trajectory are designed to effectively test the tracking capabilities and robustness of each control strategy.

Furthermore, to assess the robustness and tracking performance of the full-state feedback linearization controller under more demanding conditions, a circular reference trajectory was generated, as shown in Figure 7. Unlike the generic reference path, which is defined using cubic polynomials for both $x(s)$ and $y(s)$, the circular trajectory is generated by parametrizing the arc using an angular variable θ_s , which evolves smoothly from 0 to π radians.

A cubic polynomial is employed to interpolate the angular evolution $\theta_s(s)$, ensuring a smooth start and stop of the angular motion. The position of each point on the trajectory is then computed as:

$$\begin{cases} x(s) = R \sin \theta_s(s) \\ y(s) = R [1 - \cos \theta_s(s)] \end{cases} \quad (14)$$

where R is the radius of the circle.

This parametrization yields a semicircular path in the plane, starting at $(0,0)$ and ending at $(0,2R)$, with a continuous and smooth variation of orientation along the arc.

As with the generic trajectory, all derivatives with respect to s (and subsequently with respect to time t) are computed numerically, and a cubic time-scaling law is used to ensure that all kinematic and dynamic constraints are satisfied. The resulting reference signals for position, orientation, velocity, and steering are then used for experimental evaluation of the control algorithms.

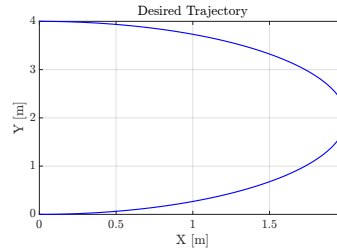


Figure 7: Circular trajectory used to evaluate the robustness of the full-state feedback linearization controller.

A circular trajectory is generally considered more challenging for trajectory tracking controllers, as it requires the robot to maintain continuous changes in curvature and orientation, imposing persistent steering and velocity adjustments. This tests the controller's ability to handle non-constant, coupled state dynamics and to maintain accuracy even during sustained, coordinated turning motions.

3 Controllers and Odometry

In this section, we provide a detailed overview of the controllers implemented for the mobile base of our robot. We also introduce the odometry sensor, which supplies information about the current position of the robot. However, since odometry is inherently affected by cumulative errors such as wheel slippage and sensor drift, we additionally implemented a SLAM (Simultaneous Localization and Mapping) system. SLAM helps to correct the inaccuracies of odometry by using environmental features to continuously update and improve the robot's estimated position.

3.1 Control via approximate linearization

The key principle behind this control method is to linearize the nonlinear tracking error dynamics of the robot around a reference trajectory. This process results in a linear time-varying system, which makes it possible to apply standard linear control strategies for trajectory tracking. Notably, even though the resulting system is time-varying, it is still possible to design the closed-loop error dynamics so that their eigenstructure remains constant over time.

In this framework, the tracking error is defined as the difference between the desired and actual state of the robot ($\tilde{x}_i = x_{di} - x_i$, $i = 1, \dots, 4$), and the system is typically transformed into chained-form coordinates to facilitate the linearization process.

The nonlinear error dynamics for the chained-form model can be expressed as follows:

$$\begin{cases} \dot{\tilde{x}}_1 = \tilde{u}_1 \\ \dot{\tilde{x}}_2 = \tilde{u}_2 \\ \dot{\tilde{x}}_3 = x_{d2}u_{d1} - x_2u_1 \\ \dot{\tilde{x}}_4 = x_{d3}u_{d1} - x_3u_1 \end{cases} \quad (15)$$

To design a tracking controller, these error equations are linearized around the desired trajectory. This process leads to a *linear time-varying* (LTV) system, where the evolution of the error vector $\tilde{x}$ depends on time-varying coefficients determined by the reference trajectory. The resulting LTV error dynamics can be written as:

$$\dot{\tilde{x}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & u_{d1}(t) & 0 & 0 \\ 0 & 0 & u_{d1}(t) & 0 \end{bmatrix} \tilde{x} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ x_{d2}(t) & 0 \\ x_{d3}(t) & 0 \end{bmatrix} \tilde{u} \quad (16)$$

where $u_{d1}(t)$ is the reference input associated with the desired trajectory.

For this linearized system, a feedback control law can be synthesized by placing the closed-loop eigenvalues (poles) in suitable locations to ensure stability and desired convergence rates. In particular, the following state feedback is typically used:

$$\begin{cases} \tilde{u}_1 = -k_1\tilde{x}_1 \\ \tilde{u}_2 = -k_2\tilde{x}_2 - \frac{k_3}{u_{d1}}\tilde{x}_3 - \frac{k_4}{u_{d1}^2}\tilde{x}_4 \end{cases} \quad (17)$$

where $k_1 > 0$ and the remaining gains k_2, k_3, k_4 are chosen such that the characteristic polynomial $\lambda^3 + k_2\lambda^2 + k_3\lambda + k_4$ is Hurwitz, meaning that all its roots have negative real parts. This guarantees that the tracking errors converge to zero under the given feedback, at least locally around the trajectory.

It is important to note that this feedback structure relies on the invertibility of the reference input u_{d1} , meaning that u_{d1} must not approach zero during motion (the reference forward velocity must remain bounded away from zero). Otherwise, during stops or sharp turns, the gains in the second feedback term may become

excessively large, leading to poor conditioning of the control law, amplification of measurement noise, and potentially destabilizing oscillations in the control input.

When the reference trajectory comes to a stop, maintaining the eigenvalues at a fixed negative value would necessitate increasingly large control gains. This is undesirable, as it can lead to impractically large control efforts and even instability near trajectory endpoints where the desired velocity vanishes.

To address this issue, we adopt an input scaling approach, whereby the closed-loop eigenvalues are chosen as functions of the reference input u_{d1} rather than as constant values. Specifically, we choose to place three coincident real eigenvalues at $-\alpha|u_{d1}|$. In this way, as the desired trajectory slows down and u_{d1} approaches zero, the control gains also diminish accordingly.

The resulting control law for the second input becomes:

$$\tilde{u}_2 = -3\alpha|u_{d1}|\tilde{x}_2 - 3\alpha^2 u_{d1}\tilde{x}_3 - \alpha^3|u_{d1}|\tilde{x}_4, \quad (18)$$

where α is a positive design parameter and in our code is indicated as k_2 .

This input scaling procedure ensures that the control input naturally tends to zero as the reference trajectory halts, avoiding the need for unreasonably high gains. Moreover, it preserves the stability of the closed-loop system, as can be rigorously demonstrated using Lyapunov-based analysis.

We adopted this velocity-scaled gain selection to improve the robustness of the controller and to better account for practical situations in which the reference velocity varies significantly along the trajectory.

Notice that the first gain k_1 , which regulates the correction along the trajectory, must often be set to a very small value in practice. If k_1 is chosen too large, the correction term $-k_1\tilde{x}_1$ can dominate the feedforward velocity u_{d1} , potentially resulting in a commanded velocity $u_1 = u_{d1} - k_1\tilde{x}_1$ that becomes negative. In such a case, the controller would attempt to command the robot to move in reverse.

The only way to guarantee that the robot does not switch to reverse motion is to select a very low value for k_1 , thereby ensuring that the correction term is always smaller in magnitude than the feedforward reference.

The approximate linearization controller performs optimally when the reference trajectory is sufficiently regular, for example, a straight line or a smooth curve with constant or slowly varying velocity and curvature. In these cases, the time-variation in the linearized system is minimal, and the assumptions underlying the controller are well satisfied. However, the method is inherently local: the guarantee of convergence applies only when the tracking error is small enough that the linear approximation remains valid. For larger deviations or when the robot starts far from the trajectory, the true nonlinear dynamics may deviate significantly from the linearized model, which can impair performance or even result in instability.

In the context of the present work, the controller was applied to a scenario involving a time-varying reference trajectory with moderate pronounced curvature. This situation inherently violates some of the key conditions for the success of the approximate linearization approach. Specifically, the reference velocity varies along the path and can become very small in regions of high curvature, which causes the feedback gains to increase dramatically. Additionally, the rapid changes in the trajectory result in a linearized model whose parameters fluctuate over time, making it more difficult for the linear controller to guarantee robust and consistent tracking.

These findings are fully consistent with the limitations documented in the literature. The approximate linearization controller, while elegant and effective in theory, delivers its best performance when applied to reference trajectories that are smooth, with nonzero and slowly varying forward velocities, and when the system operates close to the reference path. When these conditions are not met, the controller's effectiveness is diminished, highlighting the necessity for alternative, more robust control strategies when tracking complex or aggressive trajectories.

Once the corrective terms $\tilde{u}_1$ and $\tilde{u}_2$ have been determined, the actual control inputs can be computed as $u_1 = u_{1d} + \tilde{u}_1$ and $u_2 = u_{2d} + \tilde{u}_2$. These values can then be substituted into (6) in order to obtain the velocity commands to be applied to the mobile base of the robot.

3.2 Input-output linearization via static feedback

The goal of the input-output linearization via static feedback is to directly control the motion of a specific point P (not necessarily on the robot) along a desired reference path in the Cartesian plane (Figure 8). The outputs are thus selected as the Cartesian coordinates of P , i.e., $z = \begin{bmatrix} x_P \\ y_P \end{bmatrix}$, where the location of P is expressed as follows:

$$z = \begin{bmatrix} x + L \cos \theta + \Delta \cos(\theta + \phi) \\ y + L \sin \theta + \Delta \sin(\theta + \phi) \end{bmatrix} \quad (19)$$

with $\Delta \neq 0$.

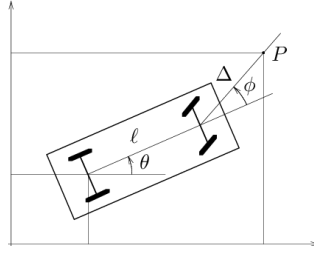


Figure 8: Alternative output definition for a car-like robot

The first step in the linearization algorithm consists in computing the time derivative of the output vector, leading to a relationship of the form:

$$\dot{z} = A(\theta, \phi)v, \quad (20)$$

where $A(\theta, \phi)$ is the decoupling matrix depending on the robot's orientation θ and steering angle ϕ , and v is the vector of velocity inputs (forward velocity and angular velocity).

For the car-like model, the matrix $A(\theta, \phi)$ is, in general, not always invertible, particularly when considering the midpoint of the rear axle as the output point. In such cases, the decoupling matrix becomes singular, making exact input-output linearization with static feedback impossible. To circumvent this issue, the output point is redefined as the point P . In this way it is possible to ensure that $A(\theta, \phi)$ remains invertible, allowing one to compute the required velocity commands by inverting the decoupling matrix:

$$v = A^{-1}(\theta, \phi)r, \quad (21)$$

this ensures that the output dynamics can be directly regulated by an auxiliary control input, typically using a simple proportional (or proportional-derivative) feedback on the tracking error.

In practice, the auxiliary input $r = \dot{z}$ is chosen as follows for each component:

$$r_i = \dot{z}_{i,ref} + k_{pi}(z_{i,ref} - z_i), \quad k_{pi} > 0, \quad i = 1, 2. \quad (22)$$

This choice guarantees exponential convergence of the output tracking error to zero for any initial configuration.

So, the tracking errors are used in the linear feedback law to compute r , and finally the velocity commands (v, ω) are found by solving the linear system via matrix inversion.

This approach exhibits several strengths. Most notably, it allows for the exact linearization of the input-output dynamics, resulting in decoupled first-order integrators for each Cartesian output. This greatly

simplifies the tracking control problem, making it possible to achieve arbitrary exponential convergence of the tracking errors through proper choice of the gains k_{pi} . The method is also conceptually simple and can be applied in real time, since it does not require the computation of higher-order derivatives or complicated nonlinear transformations.

However, several limitations must also be acknowledged. First, while the outputs (x_P, y_P) are stabilized, the internal robot variables (such as orientation θ and steering angle ϕ) are not directly regulated by the controller and instead evolve according to the so-called "internal dynamics," whose stability is not guaranteed a priori. Ensuring boundedness of these internal variables may require additional analysis or constraints on the reference trajectory.

Another practical issue is that the decoupling matrix must be inverted at every control step, which, while tractable for two-dimensional outputs, may become computationally more demanding for higher-dimensional models. Moreover, the design requires full-state knowledge (or reliable estimation) of all configuration variables.

In summary, input-output linearization via static feedback offers an elegant and effective framework for Cartesian trajectory tracking in car-like robots, allowing for precise tracking of a chosen point. Its main strengths are the simplicity of the resulting linear error dynamics and the ability to achieve fast exponential convergence. Nevertheless, care must be taken in the design and selection of outputs, in the analysis of the internal dynamics, and in handling potential singularities or numerical issues in the decoupling matrix inversion.

3.3 Full-state linearization via dynamic feedback

The full-state feedback linearization controller represents the most general and powerful approach among the linearization-based controllers for car-like robots, as it enables the exact transformation of the entire nonlinear system dynamics into a set of decoupled and fully controllable integrator chains.

This approach overcomes the singularity issues encountered in static input-output linearization, particularly when the decoupling matrix becomes singular.

The essential idea is to apply a process known as dynamic extension, in which auxiliary states and additional differentiations are introduced so that the control inputs appear explicitly in the highest-order derivatives of the chosen outputs.

In the context of trajectory tracking for a car-like robot, the natural choice for the outputs is the Cartesian coordinates of a point rigidly attached to the robot, typically the position (x, y) . However, when differentiating these outputs with respect to time, the input vector does not appear directly in all components, leading to a singular decoupling matrix. To address this, the dynamic extension procedure augments the system with new state variables (often denoted ξ_1 and ξ_2), corresponding to the first and second derivatives of the first output, and recursively differentiates the outputs until the inputs explicitly appear. The resulting augmented system can be described, after two dynamic extensions, by the following relations:

$$\begin{aligned} z_1 &= x_1, & z_2 &= x_4, \\ \dot{z}_1 &= \xi_1, & \dot{z}_2 &= x_3 \xi_1, \\ \ddot{z}_1 &= \xi_2, & \ddot{z}_2 &= x_2 \xi_1^2 + x_3 \xi_2, \\ \dddot{z}_1 &= r_1, & \dddot{z}_2 &= r_2, \end{aligned} \tag{23}$$

where x_2 and x_3 are chained-form coordinates, ξ_1 and ξ_2 are the dynamic extension variables, and r_1, r_2 are the final auxiliary control inputs designed to stabilize the tracking error for each output independently.

The design of the auxiliary inputs r_1 and r_2 typically follows a third-order error dynamics:

$$r_i = \ddot{z}_{i,ref} + k_{ai}(\ddot{z}_{i,ref} - \ddot{z}_i) + k_{vi}(\dot{z}_{i,ref} - \dot{z}_i) + k_{pi}(z_{i,ref} - z_i), \quad i = 1, 2, \quad (24)$$

with feedback gains k_{ai}, k_{vi}, k_{pi} chosen such that the associated characteristic polynomial is Hurwitz, ensuring exponential convergence of the tracking errors.

In practice, the real control inputs (v, ω) for the robot are computed through a sequence of coordinate and input transformations. The virtual inputs u_1 and u_2 are obtained as

$$u_1 = \xi_1, \quad u_2 = \frac{r_2 - x_3 r_1 - 3x_2 \xi_1 \xi_2}{\xi_1^2}, \quad (25)$$

and are then mapped to the physical commands for the mobile base by inverting the nonlinear kinematic relations.

The practical implementation in the provided code closely follows this theoretical framework. The desired outputs and their time derivatives (up to the third order) are obtained from the reference trajectory, and numerical differentiation or interpolation is employed to approximate these quantities. The system's actual state is mapped into the chained-form coordinates (x_1, x_2, x_3, x_4) , and the extended states ξ_1 and ξ_2 are updated at each control step according to the dynamic extension procedure. The error dynamics are regulated using the third-order feedback law, resulting in the auxiliary inputs r_1 and r_2 . Finally, the virtual control inputs u_1 and u_2 are computed and mapped to the robot's linear and angular velocities to be applied to the hardware or simulation by using (6).

In order to guarantee exact reproduction of the desired output trajectory from the very beginning (that is, to ensure that the tracking error is identically zero at the initial time) it is necessary to set the initial conditions of all relevant variables according to the desired reference and its time derivatives.

The state variables of the chained-form system (x_1, x_2, x_3, x_4) and the dynamic extension variables (ξ_1, ξ_2) are then set as follows:

$$\begin{aligned} x_1(t_0) &= z_{d1}(t_0) = x_d(t_0), \\ x_2(t_0) &= \frac{\dot{z}_{d1}(t_0)\ddot{z}_{d2}(t_0) - \ddot{z}_{d1}(t_0)\dot{z}_{d2}(t_0)}{\dot{z}_{d1}^3(t_0)}, \\ x_3(t_0) &= \frac{\dot{z}_{d2}(t_0)}{\dot{z}_{d1}(t_0)}, \\ x_4(t_0) &= z_{d2}(t_0) = y_d(t_0), \\ \xi_1(t_0) &= \dot{z}_{d1}(t_0), \\ \xi_2(t_0) &= \ddot{z}_{d1}(t_0). \end{aligned}$$

By setting the initial conditions in this manner, the robot and the controller are both synchronized with the desired output trajectory and its time derivatives. As a result, the output will exactly follow the reference trajectory from the very start, and no transient tracking error will occur. In any other case, for example, if the system or controller is initialized differently, a transient error will be present, but it will still converge exponentially to zero with a rate determined by the chosen feedback gains.

To sum up, this approach achieves true full-state linearization, meaning the entire closed-loop system (including both output and internal dynamics) is rendered linear and fully controllable. As a result, the controller enables precise and robust trajectory tracking with arbitrary exponential convergence rates, even in the presence of complex or rapidly varying reference trajectories. The method is also systematic and extensible to higher-dimensional systems.

However, several practical and theoretical limitations must be considered. First, the method requires the computation (and possibly numerical differentiation) of higher-order derivatives of the reference trajectory,

which may introduce noise or instability if the reference is not sufficiently smooth (this is the reason why we introduced a low-pass filter in our planner file). The dynamic extension increases the dimension of the controller’s state, potentially making the implementation more computationally demanding and sensitive to modeling errors. Moreover, the invertibility of the transformation and the well-posedness of the input computations must be guaranteed throughout the operation, especially when $\dot{\xi}_1$ (the time derivative of the x -coordinate) approaches zero, which may cause the denominator in u_2 to vanish or become ill-conditioned.

3.4 Odometry and SLAM

For the low-level actuation and odometry of the mobile robot, we decided to use the `steering_controllers_library` provided by the ROS2 Control framework. This library implements standard kinematic models and odometry for steering-based vehicles (including bicycle, tricycle, and Ackermann steering), handling the interface with hardware abstraction and providing reliable state estimation and joint command management.

The controller was configured in our `robot_ackermann_control.yaml` file to specify a four-wheeled robot with front steering and to set the controller to operate in classic (non-chained) mode, meaning that a single `TwistStamped` message is used for velocity commands rather than separate reference interfaces for velocity and steering.

The controller uses velocity feedback from the hardware interfaces, which is typically more robust than the position one in the presence of wheel slip or actuator errors.

The correct names for the front and rear wheel joints are provided, matching the URDF and hardware interfaces.

The parameters for wheelbase, wheel radii, and wheel track ensure that the kinematic model accurately reflects the geometry of the actual robot, allowing the library to compute both forward and inverse kinematics correctly.

The odometry and base frame IDs are set to align with the overall robot TF tree, ensuring compatibility with localization and navigation stacks.

The `steering_controllers_library` autonomously handles the computation and publication of odometry, joint commands, and TF transformations, which greatly simplifies the integration of custom trajectory tracking algorithms.

In this project, the high-level trajectory tracking controllers compute reference linear and angular velocities, which are then sent as `Twist` commands to the steering controller. The library translates these references into the appropriate low-level wheel velocities and steering angles, executing the required motions on the physical or simulated robot while providing feedback through standard ROS topics.

Overall, the use of this library enables rapid development and robust operation of mobile robots with steering mechanisms, abstracting the complexity of low-level actuation and state estimation, and allowing the developer to focus on higher-level control and planning strategies.

But this library proved to be extremely valuable in this project, primarily thanks to its integrated odometry functionalities.

Odometry refers to the process of estimating a robot’s position and orientation over time by integrating data from wheel encoders and steering angles. This information is essential for mobile robots, as it provides a real-time estimate of the robot’s current pose within a given reference frame.

In our implementation, odometry data provided by the steering controller served as the foundation for retrieving the robot’s position and orientation, which were then used as feedback for all high-level control algorithms and trajectory tracking controllers.

However, odometry inherently suffers from accumulating errors over time due to wheel slip, encoder noise,

and the propagation of small measurement inaccuracies. These issues can cause the robot’s estimated position to drift significantly from its true location, especially during long or complex maneuvers. In fact, we observed that the odometry alone was prone to significant drift, as evidenced by the mismatch between the odometry-reported pose and the actual robot position shown in the Gazebo GUI. To mitigate this limitation, we integrated Simultaneous Localization and Mapping (SLAM) techniques into our system using the `slam_toolbox` package.

SLAM is a fundamental technique in mobile robotics that allows a robot to build a map of an unknown environment while simultaneously determining its own pose within that map. In our application, this process relies on laser scan data from the robot’s lidar sensor and pose estimates derived from odometry. The localization output is the real-time transformation between the map frame and the odom frame, which is continuously updated as the robot moves and collects new sensor data.

The implementation uses two main operational modes of `slam_toolbox`: mapping and localization. In mapping mode, the robot explores the environment and incrementally constructs a 2D occupancy grid map, which is published on the `/map` topic and can be visualized in RViz. In localization mode, the system uses an existing map and focuses on estimating the robot’s pose within it by matching incoming laser scans to the stored map.

Configuration is managed through dedicated YAML parameter files: `"mapping.yaml"` for mapping mode and `"localization.yaml"` for localization mode. These files set all relevant SLAM parameters, such as frame IDs (`odom_frame`, `map_frame`, `base_frame`), scan topic, map resolution, scan matching algorithms, loop closure options, and thresholds for minimum travel distance and heading changes required to update the pose estimate.

The launch file is designed to flexibly select between mapping and localization at runtime. This modular approach allows seamless switching between mapping a new environment and localizing within an existing one, simply by changing launch arguments.

So, by leveraging SLAM, we ensured that our robot maintained an accurate and consistent estimate of its position over time, which was fundamental for the effectiveness and reliability of all implemented controllers.

4 Comparison and discussion of controllers’ performance

4.1 Control via approximate linearization results

This section analyzes the system behavior under approximate linearization. This controller is expected to perform worse overall compared to the other two discussed later. This is because it is not well suited for more complex trajectories—such as the curvilinear path considered in this case. In fact, the controller is better suited for simpler systems with slow-time dynamics. However, the relatively conservative limits imposed by the planner on the control inputs help to mitigate the severity of the observed issues.

The test was conducted with the following gains: $k_1 = 0.001$, $k_2 = 0.2722$.

As shown in Figure 9, the system behavior is unsatisfactory: the robot follows the path with a significant lateral offset. This issue becomes even more evident when analyzing the temporal evolution of the tracking errors along the two reference axes, as illustrated in Figure 10.

Specifically, the error along the x -axis (e_x) exhibits an initial peak of nearly 1 m, which gradually decreases and stabilizes at a steady-state offset of approximately 0.2 m. Conversely, the error along the y -axis (e_y)

shows a negative peak close to -1 m during the final phase of the trajectory, corresponding to the deceleration phase. However, this error is quickly compensated.

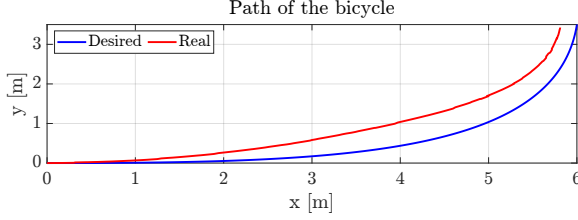


Figure 9: Real vs desired path

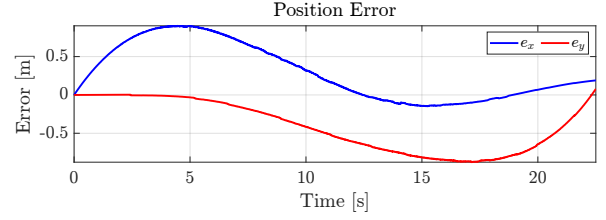


Figure 10: Time evolution of position error

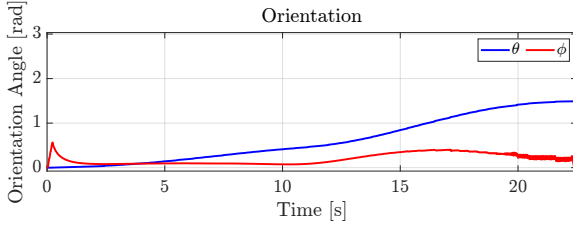


Figure 11: Time evolution of orientation.

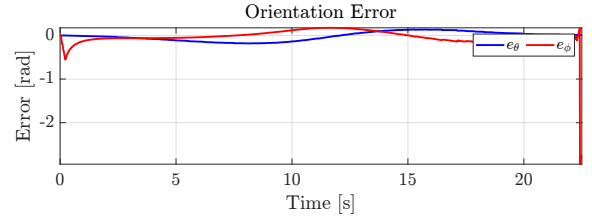


Figure 12: Time evolution of orientation errors.

The orientation control can then be analyzed, as shown in Figures 11 and 12. Regarding orientation control, the system exhibits significantly better performance. The orientation error, both in terms of the heading angle θ and the steering angle ϕ , remains consistently low throughout most of the trajectory.

However, a spike for ϕ is observed near the end of the trajectory. Although this behavior is evident in the data, it has no practical impact on the simulation, as it occurs when the linear velocity is nearly zero. It is likely caused by the controller's attempt to compensate for a residual error along the x -axis.

Specifically, due to the definition of the virtual control input provided in equation 18, this input tends to vanish when the x -axis error becomes zero. Consequently, if a non-zero error along the x direction is still present near the end of the trajectory, the controller may apply a more aggressive corrective action, which manifests as the observed spike.

Despite the generally good performance, the magnitude of this final spike indicates that the controller's behavior cannot be considered entirely satisfactory.

Figures 13 and 14 illustrate the evolution of control inputs v and ω . To keep the actuator efforts for v and ω within reasonable limits the reference trajectory generated by the planner was designed to strictly respect maximum values of 0.5 m/s for the linear velocity and 2 rad/s for the angular velocity. These maximum values are comfortably within the physical capabilities of our robot.

It is important to note, however, that although the planner ensures the reference trajectory complies with these bounds, such constraints are not automatically enforced on the actual control inputs applied to the robot. Without explicit saturation mechanisms within the controller, tracking errors or aggressive corrective actions may cause the commanded values of v and ω to exceed the planned limits. For this reason we implemented this saturation within the control law to guarantee that the actuator commands remain within the prescribed bounds.

Now, we can see that the velocity starts and ends at zero and never exceeds the limit of 0.5 m/s. Moreover, it is particularly smooth and free of oscillations. The absence of sudden accelerations or decelerations indicates that the controller is able to generate physically plausible commands that are compatible with the robot's actuation constraints.

Regarding the control input ω , the results align with the analysis of the evolution of ϕ : for most of the trajectory, ω remains within low values, never exceeding 0.2 rad/s. Only at the end of the trajectory are negligible oscillations observed, followed by a spike of approximately -0.8 rad/s, which is still less than half of the saturation limit of -2 rad/s.

In conclusion, although the control inputs exhibit positive traits such as the smoothness of the linear velocity v and the limited magnitude of spikes in the angular input ω , the overall system behavior cannot be considered fully satisfactory. The presence of residual oscillations and observed peaks, although contained, indicates that there is still room for improvement in tuning the control parameters to achieve a more stable and accurate response throughout the entire trajectory.

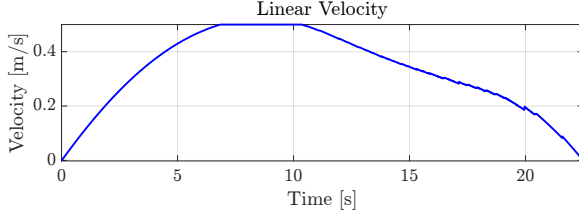


Figure 13: Time evolution of linear velocity.

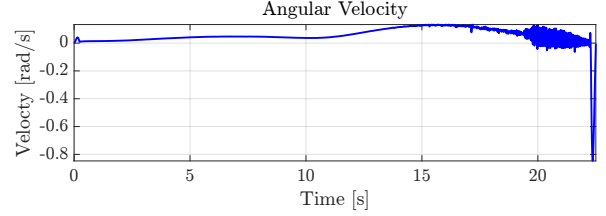


Figure 14: Time evolution of angular velocity.

4.2 Input-Output linearization via static feedback results

In this section, we analyze the system's behavior under Input-Output Feedback Linearization.

In the implementation of this controller, we found it unnecessary to apply explicit saturation, as the values of v and ω never exceed the imposed limits.

The results presented in the following plots were obtained with $\Delta = 0.2$, $k_1 = 0.2$, and $k_2 = 0.15$.

Figure 15 presents a comparison between the desired and the actual path followed by the robot under the input-output feedback linearization controller.

While the two paths seem almost coincident, particularly in the initial phase, a detailed analysis of the time evolution of the position errors (Figure 16) reveals the presence of a substantial offset along the entire trajectory. In particular, the error along the x -direction progressively decreases, whereas the error along the y -direction increases and almost appears to diverge over time.

As shown in the plot, the maximum tracking error e_y occurs shortly before reaching the final position, specifically during the deceleration phase. The peak error reaches approximately -0.4 m along the y axis. Throughout the rest of the trajectory, the tracking error remains relatively low and within acceptable bounds. The behavior of e_y is typical for tracking controllers when the robot approaches the end of the path and needs to decelerate rapidly, often resulting in increased errors due to both control limitations and actuator constraints.

Conversely, the error along the x axis exhibits a peak slightly above 0.6 m during the initial part of the trajectory, corresponding to the acceleration phase, whereas it becomes negligible during the final part of the trajectory.

This behavior can be attributed to several factors. Firstly, it is important to recall that this type of control does not act directly on the robot's orientation θ or the steering angle ϕ , but rather on the position of a virtual point offset from the origin of the front wheel of the bicycle along the direction defined by the steering angle ϕ . However, since in practical scenarios the objective is to control the robot itself rather than the virtual point P , the evolution of the `robot_base_footprint` frame is shown in the plots. In conclusion, the tracking performance is influenced by both the choice of the offset and the internal dynamics of the system.

The lack of direct control over the orientation angles is far from a minor issue, particularly given that our mobile manipulator is a non-holonomic system. Since the robot does not employ omnidirectional wheels, the only way to achieve lateral or arbitrary motion is by appropriately steering the wheels and, consequently, reorienting the entire mobile base. This means that the system's ability to reach a desired position or follow a complex trajectory critically depends on its capacity to control and adjust its orientation. As a result, neglecting the orientation dynamics can severely limit the maneuverability and overall effectiveness of the mobile platform, especially when precise or agile movements are required.

The evolution of the angles θ and ϕ , as well as the corresponding tracking errors, are not reported in the following analysis, since these variables are not directly controlled and thus do not provide meaningful insight into the controller's actual performance.

To demonstrate the exponential convergence of the system, an additional test was performed by extending the trajectory duration by 5 seconds. In this extended interval, the desired position remains fixed at its final value, allowing the controller sufficient time to reduce the residual position error along the y axis, as shown in Figure 16. The plot clearly exhibits a decreasing exponential trend in the error. Although the convergence is relatively slow, the residual error remains negligible, making the overall performance acceptable. As usual, a faster convergence could be achieved by increasing the controller gains, at the cost of introducing higher oscillations in the control inputs.

This test also highlights one of the most favorable features of the controller: there is no smoothness requirement for the trajectory of point P , and consequently for the bicycle model itself. This flexibility simplifies the generation of feasible trajectories, especially in scenarios where sharp changes in direction or velocity may occur.

Further tests confirmed that the system's performance remains largely unchanged for variations of Δ even up to one order of magnitude. However, for excessively large values of Δ , a significant degradation in tracking performance is observed, as illustrated in the right plot of Figure 15.

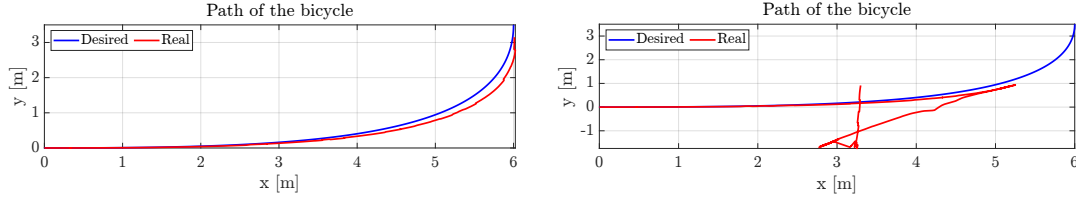


Figure 15: Comparison between the real (red) and desired (blue) path for two different values of the offset parameter: (left) $\Delta = 0.2$, (right) $\Delta = 20$.

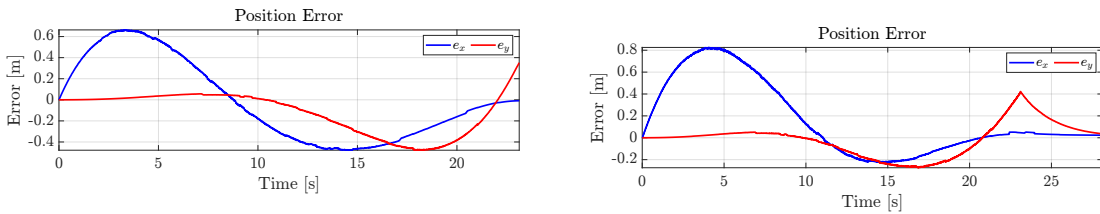


Figure 16: Comparison of the time evolution of the the normal trajectory (left) and the enlarged trajectory (right).

Figure 17 illustrates the evolution of the control inputs v and ω . As can be observed, the maximum value of v remains just below 0.5 m/s, while ω slightly exceeds -0.4 rad/s. The linear velocity profile is continuous and smooth throughout the trajectory, starting from zero and reaching a non-zero but relatively low value at the end. As for the angular velocity, it remains continuous or exhibits only negligible oscillations for most of the motion. A single spike is observed at the very end of the trajectory; however, this anomalous behavior does not affect the system's performance, as clearly visible in the video.

This is because the variation in ω , and consequently in the steering angle ϕ , occurs when the vehicle is nearly stationary and remains within acceptable bounds, never approaching actuator saturation. Therefore, the final result can be considered more than satisfactory for the scenario under analysis.

It is worth noting that the magnitude and smoothness of the control inputs directly depend on the tuning of the feedback gains. While the results here are satisfactory, different gain values may yield faster convergence at the cost of increased control effort or potential oscillations.

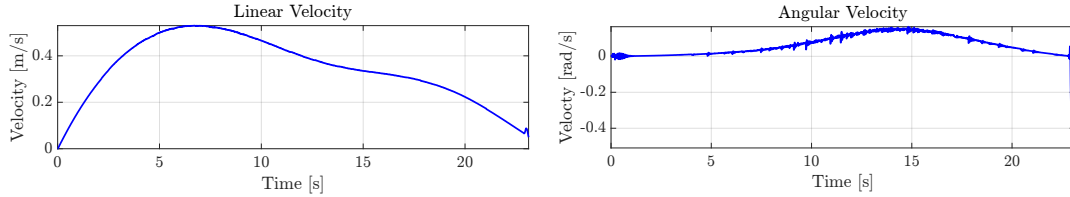


Figure 17: Control inputs.

4.3 Full-state linearization via dynamic feedback results

This paragraph analyzes the performance of the controller designed via full-state linearization with dynamic feedback. Based on the characteristics of this control approach, and as discussed in the previous sections, improved tracking performance is expected.

Initially, the controller's behavior is evaluated for the reference path shown in Figure 6. To further assess the robustness of this control strategy, an additional test is conducted on a circular trajectory (Figure 7).

It is important to note that the chained form model adopted in the control design exhibits an intrinsic singularity when $\theta = \frac{\pi}{2}$. To mitigate the risks associated with this configuration, a saturation block was introduced in the implementation. While this mechanism cannot mathematically guarantee that the system will never approach the singularity, it does serve to limit the magnitude of the control inputs when θ is near the critical value. In practice, this reduces the likelihood of generating excessively large or unfeasible commands that could potentially damage the robot or compromise its safety. Thus, although the saturation cannot fully prevent the occurrence of the singularity, it provides an important layer of protection for the hardware.

All simulations were carried out using the following gain values: $k_{a_{1,2}} = 0.5$, $k_{v_{1,2}} = 2.0$, and $k_{p_{1,2}} = 0.65$. In Figure 18 we can observe that, as expected, the controller is able to accurately follow the desired trajectory, with the actual path closely matching the reference curve for the entire duration of the experiment. The position error remains negligible throughout the motion, demonstrating the effectiveness of the full-state feedback linearization approach in ensuring precise trajectory tracking. This high level of performance is further supported by the absence of any initial tracking error, which allows the mobile manipulator to maintain close adherence to the planned path from the outset.

Overall, the results confirm that, at least for this scenario, the controller provides excellent position tracking performance.

An even more compelling confirmation of the excellent performance achieved with this control approach can be observed by examining the overall trend of the tracking errors in the x and y components, as shown in Figure 19.

Both the x and y tracking errors remain consistently low for the entire duration of the trajectory. The error in the x component fluctuates around 0.04 m, while the y error stays below 0.02 m for most of the trajectory. These values are extremely small compared to the scale of the motion and confirm the high accuracy of the controller.

The slight fluctuations observed in the error signals can be attributed mainly to the discretization of the reference trajectory and the numerical approximations inherent in the implementation. Nonetheless, the errors remain well within acceptable bounds, and their amplitude is, in fact, particularly small.

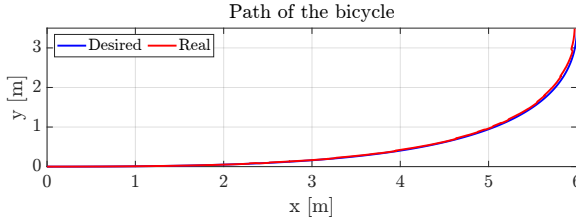


Figure 18: Real vs desired path

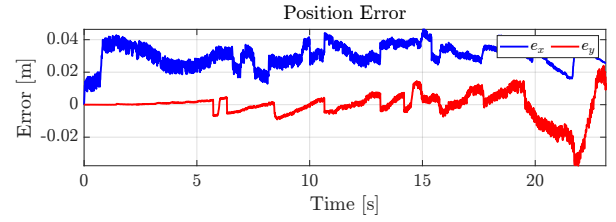


Figure 19: Time evolution of position error

Now we can focus our attention on the evolution of the orientation angles (Figure 20) and on their tracking errors (Figure 21).

As can be seen in the first plot, the evolution of the vehicle's orientation θ is smooth and continuous throughout the trajectory, while the steering angle ϕ remains well within reasonable bounds, reaching a maximum value slightly below 0.5 rad. This regular behavior reflects the effectiveness of the controller in generating feasible reference profiles for both orientation variables. Moreover, the amplitude of the steering angle ϕ remains consistently low throughout the trajectory, staying well away from the singularity condition at 90° . This ensures smooth and safe motion of the vehicle.

Regarding the orientation errors, the second plot shows that the maximum error in θ (e_θ) is about 0.25 rad, whereas the error in the steering angle (e_ϕ) peaks at approximately 0.2 rad. Both errors exhibit a clear tendency to converge towards zero over time, a trend that is particularly evident for e_θ . This demonstrates that, despite the nontrivial nature of the tracking task, the controller is able to ensure satisfactory convergence of the orientation variables.

It is important to emphasize that, due to the use of full-state linearization via dynamic feedback, the controller requires the computation of high-order derivatives (up to the third order) of the reference trajectory. As discussed previously, relying on such high-order derivatives may introduce several challenges, including noise amplification and increased sensitivity to discontinuities or sharp corners in the reference path.

To mitigate these issues, a low-pass filter was applied to the computed derivatives. However, this introduces an inevitable delay in the system's response, which limits the controller's ability to react instantaneously to changes in the reference trajectory. As a result, some lag between the desired and actual orientation variables is expected, particularly for the steering angle ϕ , which is more sensitive to rapid variations in the trajectory.

Overall, the results confirm that the proposed control strategy is able to guarantee good convergence and regular evolution of the orientation variables, provided that the reference trajectory is sufficiently smooth and well-behaved.

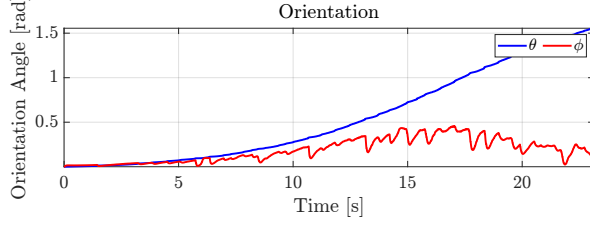


Figure 20: Time evolution of bicycle orientation

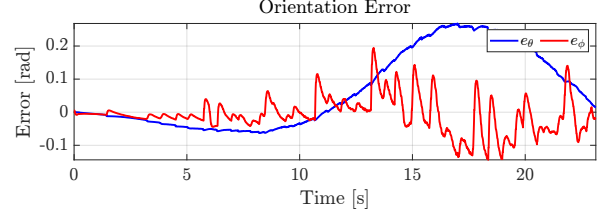


Figure 21: Orientation Error

We can now proceed to the analysis of the control inputs (Figure 22 and Figure 23).

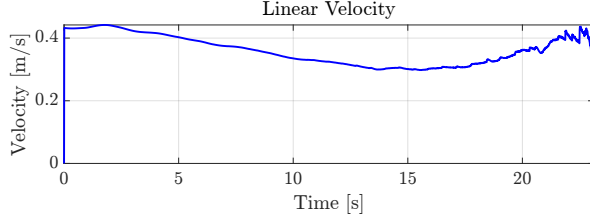


Figure 22: Time evolution of the linear velocity.

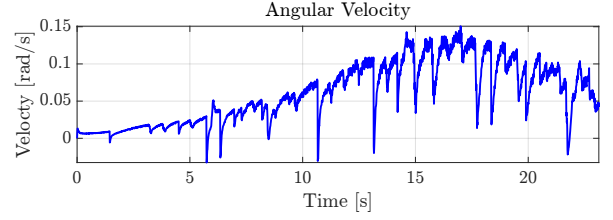


Figure 23: Time evolution of the angular velocity.

As can be observed, the linear velocity remains nearly continuous throughout most of the trajectory. However, it neither starts nor ends at zero.

This behavior does not negatively affect the simulation, as the planner enforces time-limited constraints on the control inputs, which are compatible with the system's dynamics, as clearly visible in the video.

Nonetheless, in environments characterized by stricter or different dynamic constraints, such behavior could potentially lead to undesirable effects.

Both the linear and angular velocities remain comfortably within the bounds imposed by the planner. In particular, as anticipated from the analysis of the evolution of the steering angle ϕ , the amplitude of the angular velocity remains limited throughout the trajectory, with a peak slightly below 0.15 rad/s, well under the imposed limit.

The angular velocity, however, exhibits some oscillations. These are characterized by a relatively low amplitude and were therefore deemed acceptable during the tuning phase. Further testing confirmed that a reduction in the control gains—specifically $k_{v_{1,2}}$ —would lead to a decrease in these peaks, but at the cost of degraded tracking performance. For this reason, a balanced compromise was chosen between noise reduction and tracking accuracy.

Ultimately, the transient fluctuations do not compromise the overall stability or tracking accuracy of the system, nor do they pose a risk of actuator saturation. In practical terms, this confirms the robustness and feasibility of the proposed control strategy, even when tracking challenging or highly dynamic trajectories.

To further evaluate the robustness of the controller, an additional experiment was performed using a circular trajectory, with all gain values kept unchanged. The results are presented in Figure 24 and Figure 25.

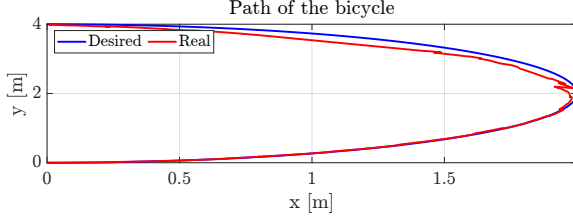


Figure 24: Real vs desired path

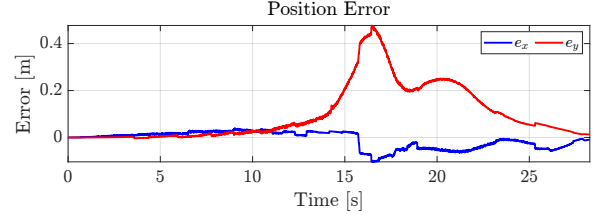


Figure 25: Time evolution of Position Error

As anticipated, when the system approaches the singular configuration at $\theta = \frac{\pi}{2}$, the control inputs reach their saturation limits. This makes it challenging for the robot to accurately follow the desired path during intervals in which θ is close to this singularity.

Nevertheless, the figures clearly demonstrate the robust and exponentially stable behavior of the controller: although the singularity causes an observable tracking error—specifically, the error along the y -axis increases from an almost negligible value to a peak of approximately 0.45 m at $t = 17$ s—the system is able to recover rapidly, and the error converges exponentially to a negligible value.

So, we can say that the controller demonstrates rapid exponential convergence even in the presence of significant initial tracking errors, as evidenced by the system’s ability to recover swiftly after encountering the singularity.

It should be emphasized that this anomalous behavior stems from the inherent structural limitations of the chained-form representation near the singular configuration. Overall, the results confirm that the controller is not only exponentially stable, but also remarkably robust in the presence of nonzero initial errors and the challenges posed by system singularities.

5 Look-At-Point behavior in manipulator control

Regarding the control of the robotic arm, our approach begins with a position control phase. At the start of the simulation, the arm is commanded to move smoothly from its initial joint configuration to a predefined “wake-up” or ready position. This is achieved by generating a trajectory, using a cubic or linear profile, from the current joint angles to the desired final configuration, and sending the corresponding position commands to the arm.

During this phase, the arm remains in the ready position, holding it precisely through continuous position commands, until an ArUco marker is detected within the camera’s field of view. This ensures that the arm starts from a controlled, repeatable pose, which is essential for reliable visual servoing and for subsequent transitions to more dynamic control modes, such as the look-at-point visual servoing.

To facilitate smooth transitions between position and velocity control modes, a dedicated function was implemented to interface with the ROS 2 controller manager. This function allows the system to dynamically activate or deactivate specific controllers according to the current task requirements, for example, switching from position control to velocity control as soon as the ArUco marker is detected, and reverting back if the marker is lost. The switching is performed asynchronously and transparently, without disrupting the ongoing control process, thus ensuring the arm can autonomously adapt its behavior in real time based on the operational context. This mechanism enables the robot to maintain both safety and task performance by always employing the most appropriate control strategy for the current situation.

At this point, thanks to the availability of the pre-existing, open-source ArucoSimple node, which subscribes to the camera feed, identifies ArUco markers in the images, and publishes their precise 3D pose with respect to the chosen reference frame, we were able to proceed with the implementation of the look-at-point behavior. Once the ArUco marker was detected, the node provided real-time pose estimates of the marker, making it possible for the robotic arm's control system to continuously orient the camera (end-effector) toward the marker even while the mobile base moves.

The objective is to ensure that the camera's optical axis always points toward the detected marker. Mathematically, the look-at-point control is achieved by regulating the direction vector $\mathbf{s} \in \mathbb{S}^2$, defined as the unit vector connecting the camera frame origin to the marker position in the camera frame:

$$\mathbf{s} = \frac{\mathbf{p}_o}{\|\mathbf{p}_o\|}, \quad (26)$$

where $\mathbf{p}_o$ is the position of the observed point (i.e., the ArUco marker) expressed in the camera frame. The goal is to align this vector $\mathbf{s}$ with a desired direction, typically the camera's optical axis, represented by $\mathbf{s}_d = [0, 0, 1]^T$.

To achieve this, we implemented the following velocity-level control law for the robot arm's joints:

$$\dot{\mathbf{q}} = k(LJ_e)^\dagger(\mathbf{s}_d - \mathbf{s}) + N\dot{\mathbf{q}}_0, \quad (27)$$

where:

- $k > 0$ is a proportional gain,
- J_e is the arm's end-effector geometric Jacobian,
- L is the interaction matrix that relates end-effector velocities to changes in the direction vector $\mathbf{s}$,
- $(LJ_e)^\dagger$ denotes the (damped) pseudoinverse of the product LJ_e ,
- $N = I - (LJ_e)^\dagger(LJ_e)$ projects any motion into the null-space of the primary task, allowing secondary objectives (such as joint limit avoidance or posture optimization) to be accomplished via $\dot{\mathbf{q}}_0$.

The interaction matrix $L(\mathbf{s})$ is given by:

$$L(\mathbf{s}) = \begin{bmatrix} -\frac{1}{\|\mathbf{p}_o\|}(I_3 - \mathbf{s}\mathbf{s}^T) & \mathbf{S}(\mathbf{s}) \end{bmatrix} R, \quad (28)$$

where $\mathbf{S}(\mathbf{s})$ is the skew-symmetric matrix of $\mathbf{s}$, and R is the current rotation matrix of the camera (extracted from the end-effector pose).

This control structure ensures that the time derivative of the direction vector, $\dot{\mathbf{s}}$, drives $\mathbf{s}$ exponentially toward $\mathbf{s}_d$.

In our implementation, as soon as the system detects the presence of the ArUco marker (via the `/aruco_single/pose` topic), a service call is made to switch the robot's arm controller from position mode to velocity mode, enabling direct velocity commands to the joints for visual servoing. The control loop then performs the following steps at each iteration:

1. **Update Kinematics:** The current joint positions and velocities are used to update the arm's kinematic chain and compute the end-effector pose and Jacobian.
2. **Compute Direction Vector:** The position of the marker relative to the camera is normalized to obtain $\mathbf{s}$.
3. **Formulate the Interaction Matrix:** The matrix $L(\mathbf{s})$ is computed as described above, leveraging the camera's current orientation.

4. **Compute Task Jacobian and Pseudoinverse:** The product LJ_e and its pseudoinverse are computed to solve for the required joint velocities.
5. **Null-Space Projection:** The null-space component is added if needed, to enforce secondary objectives or stability.
6. **Apply Joint Velocities:** The resulting joint velocity command $\dot{\mathbf{q}}$ is sent to the arm's velocity controller, ensuring the end-effector aligns its optical axis with the target at all times.

A watchdog mechanism monitors the continued visibility of the marker. If the marker is lost for a specified period (e.g., 1 second), the controller safely switches back to position mode to hold the arm in a neutral pose, thus avoiding uncontrolled or erratic motions.

This look-at-point behavior provides several advantages:

- **Continuous Visual Tracking:** The robot can maintain sight of objects of interest while moving, essential for active perception tasks.
- **Redundancy Resolution:** The controller leverages the arm's extra degrees of freedom to optimize posture or avoid singularities, thanks to the null-space component.
- **Safety and Robustness:** Automatic controller switching ensures safe operation in the presence or absence of the marker.

However, challenges include sensitivity to noise in the marker pose estimation, potential kinematic singularities, and the need for smooth controller transitions. Advanced versions can integrate visual filtering or secondary objectives for even more robust performance.

5.1 Results

For the analysis of the look-at-point behavior, we chose to focus on the case where full-state linearization via dynamic feedback is employed, as this approach yields the smoothest and most accurate trajectory. Nevertheless, simulations conducted with the other two controllers produced comparable performance in terms of manipulator control, and for this reason, their results are not reported here.

To demonstrate the correct operation of the manipulator control, we can analyze the time evolution of the orientation error between the current direction of the end-effector (computed with respect to the visual target) and the desired direction, typically chosen as the z -axis of the camera frame (Figure 26).

This error is computed as follows:

$$\theta = \arccos(\vec{s} \cdot \vec{s}_d)$$

where $\vec{s}$ is the normalized direction vector from the robot to the target, and $\vec{s}_d = [0, 0, 1]^T$ is the desired direction.

An angular error of 0 radians indicates perfect alignment, while increasing values correspond to greater misalignment. In particular, a value close to π radians means that the end-effector is oriented in the direction opposite to the target.

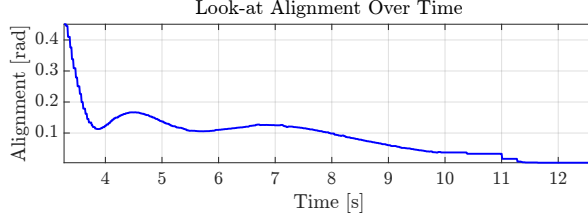


Figure 26: Angular Error (Look-at)

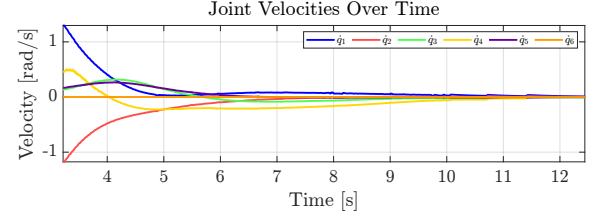


Figure 27: Time evolution of Joint velocities

As can be observed, the system starts with a relatively high alignment error (above 0.4 radians), indicating a significant initial misalignment between the camera and the marker. As the control law is applied, the alignment error decreases rapidly, demonstrating the controller’s effectiveness in driving the camera axis toward the marker. Minor fluctuations in the error are visible during the convergence phase, but the overall trend is a steady and monotonic reduction. Within a few seconds, the alignment error drops below 0.1 radians and eventually converges close to zero, indicating that the end-effector (camera) is successfully oriented toward the marker and is able to maintain this alignment over time, despite possible measurement noise or small disturbances.

This plot confirms that the implemented visual servoing strategy is able to robustly and efficiently regulate the camera’s orientation, ensuring continuous and accurate tracking of the visual target throughout the motion.

Achieving reliable look-at-point control is challenging due to the nature of the trajectory to be followed (see Figure 6). For much of the path, the robot views the ArUco marker at an angle, which compromises its accurate detection. This difficulty is further compounded during acceleration and deceleration phases, which make it harder to maintain alignment with the target. As a result, the error increases again during these intervals, but is progressively attenuated and eventually stabilizes at negligible levels.

It should be noted that, in order to avoid repeated and unnecessary activations of the switching function, a one-second threshold of marker loss was chosen before re-triggering the function.

The Figure 27, instead, shows the time evolution of the six joint velocities of the robotic arm during the look-at-point task. At the beginning of the look-at control phase, the joints exhibit relatively large velocity peaks as the arm quickly reorients itself to align the camera’s optical axis with the detected ArUco marker. These initial transients reflect the system’s rapid response to the onset of visual servoing, as the controller works to minimize the alignment error between the current camera direction and the target direction.

Shortly after this initial phase, all joint velocities decrease rapidly and stabilize near zero. This behavior indicates that the arm successfully reaches and maintains the desired alignment, requiring only minimal joint motion to compensate for small disturbances or noise in the marker pose estimation. The smooth decay and subsequent stabilization of joint velocities demonstrate both the effectiveness and stability of the velocity-level control law implemented for the look-at-point behavior. No significant oscillations or instability are observed after convergence, confirming the controller’s ability to smoothly regulate the arm motion and avoid excessive or unsafe joint commands.

Overall, the results indicate that the look-at point behavior performs well. The robot is able to correct its orientation efficiently and reach a stable aligned configuration in a short time. The joint velocity profiles further confirm that the system responds dynamically at first but then smoothly transitions to a steady state, ensuring both effectiveness and safety in the execution of the task.

Conclusion

This technical report presented the modeling, planning, and control of a car-like mobile manipulator equipped with a 5-DoF articulated arm, focusing on both trajectory tracking and visual servoing tasks. Through the implementation and experimental evaluation of three distinct control strategies—approximate linearization, input-output linearization via static feedback, and full-state linearization via dynamic feedback—we were able to critically assess their respective strengths, limitations, and practical implications.

Our comparative analysis demonstrated that, while the approximate linearization controller offers satisfactory performance for simple, slowly-varying trajectories, its effectiveness diminishes in scenarios involving more complex paths or rapid velocity changes. The input-output linearization controller, by directly regulating the position of a specific output point, enables precise tracking of the selected point but does not guarantee control over the robot’s orientation, which may limit its applicability in tasks requiring agile or omnidirectional maneuvers.

The full-state linearization via dynamic feedback, on the other hand, proved to be the most versatile and robust approach. It achieved excellent tracking accuracy even in challenging conditions and ensured the exponential convergence of both position and orientation errors. Despite its higher computational complexity and the need for careful trajectory smoothing, this method demonstrated superior performance in both standard and demanding test cases, including scenarios with near-singular configurations.

In addition to trajectory tracking, the implementation of the look-at-point behavior for the manipulator arm highlighted the system’s capability to perform coordinated visual tasks in dynamic environments. The use of velocity-level control, combined with robust controller switching and visual feedback, allowed the end-effector to maintain alignment with visual targets in real time, ensuring both task effectiveness and operational safety.

Overall, the work confirms the effectiveness of advanced control techniques for non-holonomic mobile manipulators and underscores the importance of integrating robust planning, real-time feedback, and adaptive control strategies.

References

- [1] A. De Luca, G. Oriolo, C. Samson, *Robot Motion Planning and Control*, Jean-Paul Laumond, pp. 16-35.
- [2] A. V. Pesterev, *A Linearizing Feedback for Stabilizing a Car like Robot Following a Curvilinear Path*, Pleiades Publishin, Journal of Computer and Systems Sciences Internationa, 2013, Vol. 52, No. 5, pp. 819-821.
- [3] F. Ruggiero *FIELD AND SERVICE ROBOTICS*, pp. 22-97.
- [4] M. Selvaggio *Robotics Lab*, pp. 112-116, 173-177.
- [5] During the preparation of the manuscript, the authors used ChatGPT (OpenAI) in order to improve the English language and readability. After using this tool, the authors reviewed the content and took full responsibility for the originality, validity, and integrity of the publication.